Efesto

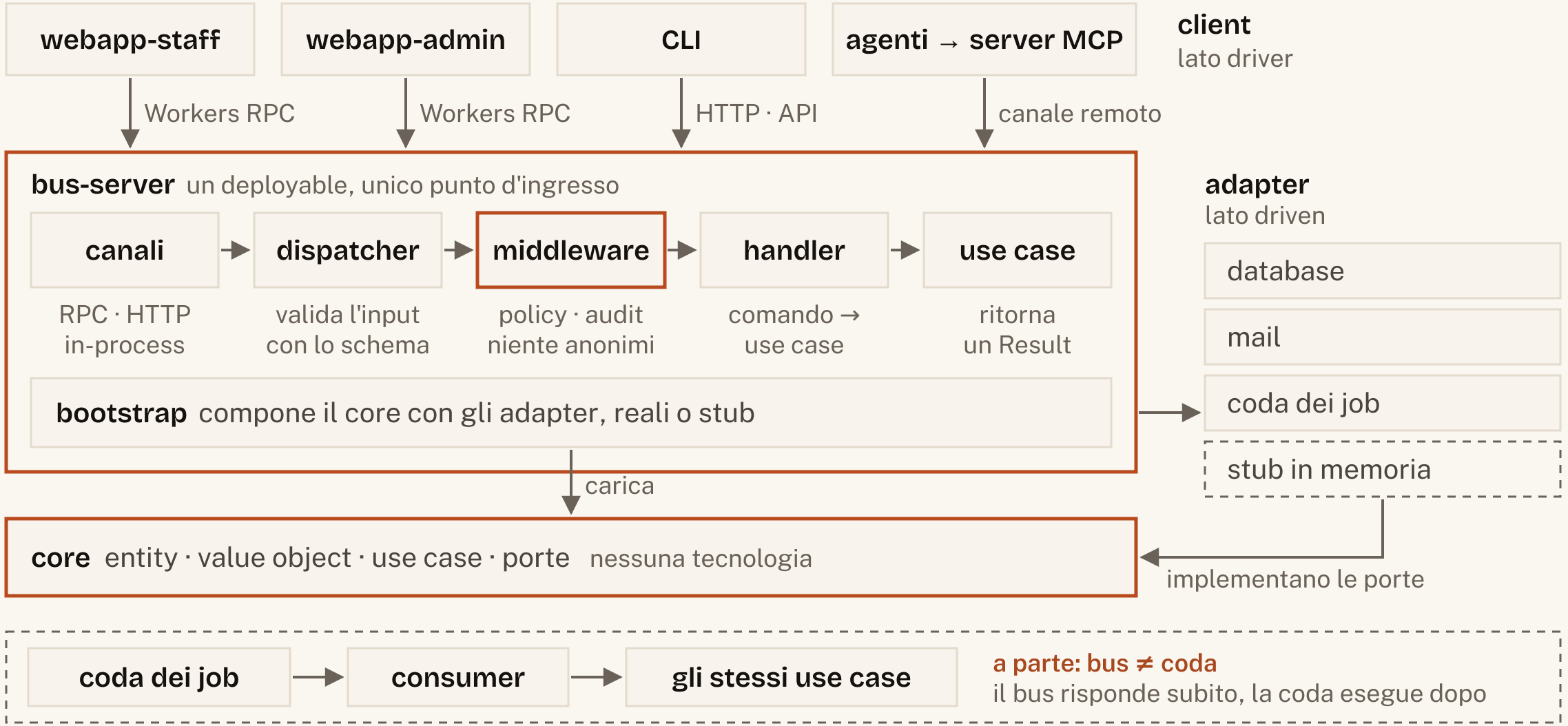
Il **blueprint** di Efesto

Come costruiamo un'applicazione perché resti elastica: le forme che prende crescendo e le parti di ciascuna.

30 ottobre 2026 · documento di lavoro, in revisione

Dove può **arrivare** un progetto

Efesto



Efesto

È il punto d'arrivo possibile, non il punto di **partenza**

1

Si parte in basso

Dal gradino più basso che basta al progetto.

2

Compare il segnale

Un bisogno concreto che il gradino attuale non regge.

3

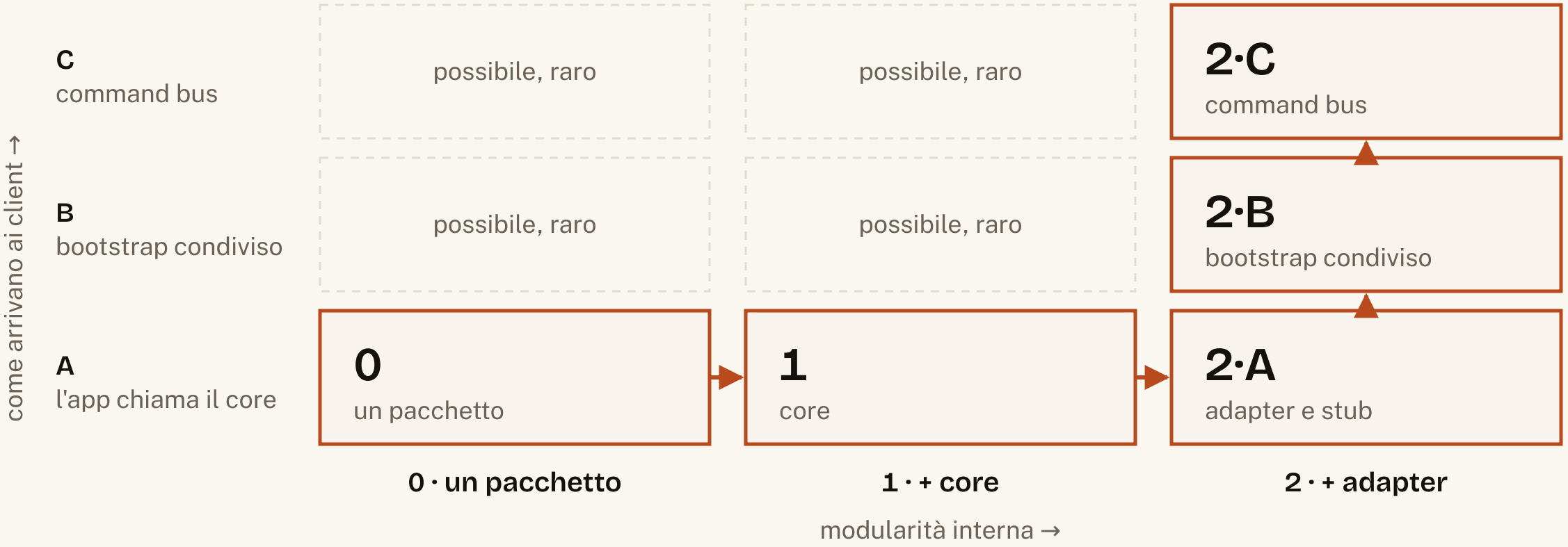
Si fa la mossa

Un refactor verso il gradino dopo. Niente riscrittura.

> *Si sale senza riscrivere solo se le regole dello use case valgono dal primo giorno.*

Efesto

Due assi, cinque **gradini**



Prima si separa la logica (0 → 1 → 2), poi cambia il modo in cui arriva ai client (A → B → C).

Un pacchetto: tutto sta **nell'app**

```
// app/routes/expenses.$id.tsx
export async function loader({ params }) {
    const expense =
        await db.expense.findUnique({
            where: { id: params.id },
        });
    if (!expense) throw notFound();
    return { expense };
}
```

→ **Per:** proof of concept, applicativi piccoli e autocontenuti.

→ **Pacchetti:** uno. Route, logica e query nello stesso posto.

→ **Limite:** la logica non si riusa e non si testa senza database.

Efesto

La logica esce dall'app ed entra nel **core**

```
// webapp: il loader chiama lo use case
const result = await context
    .resolveUseCase("expenses.GetExpense")
    .execute({ expense_id: params.id },
             { actor: context.actor });
return toResponse(result);
```

- → **Segnale:** la logica serve a un secondo consumatore (un job, una CLI) o supera la prima schermata.
- → **Mossa:** la logica passa per intero in core; i loader chiamano use case.
- → **Limite:** le implementazioni delle porte stanno nel core e si portano dietro l'ORM.

Ogni porta ha un adapter reale e uno **stub**

```
// webapp/app/container.server.ts
container.load(
    env.MOCK
        ? installStub()
        : installDb({ db: env.DB }),
    installMail({ from: env.MAIL_FROM }),
);
```

→ **Segnale:** test senza infrastruttura, un mock navigabile prima del backend, un fornitore da poter cambiare.

→ **Mossa:** le implementazioni vanno in `adapters/*`, si aggiunge lo stub, l'app sceglie cosa caricare.

→ **Limite:** il wiring vive nell'app; un secondo processo dovrebbe ricopiarlo.

Più processi, un **bootstrap** con un profilo ciascuno

```
// webapp-admin/workers/app.ts
const resolveUseCase =
    await bootstrapAdmin(env, {
        mock: env.MOCK === "1",
        requires: ["DB", "MAIL_FROM"],
    });
```

→ **Segnale:** un'altra webapp, un consumer di code, un cron sullo stesso core.

→ **Mossa:** il composition root diventa il pacchetto `bootstrap`. Un binding mancante è un errore con nome.

→ **Limite:** ogni processo controlla i permessi per conto suo: le stesse regole, scritte più volte.

Gli use case esposti diventano **comandi**

```
// webapp: chiama il bus, non il core
const result = await bus.dispatch(
    "approveExpense",
    { expense_id },
);
```

- → **Segnale:** client che non ospitano il core (CLI, MCP, agenti); regole d'accesso da riscrivere in ogni webapp.
- → **Mossa:** per ogni use case esposto, un comando con la policy e un handler.
- → **Costo:** un salto di rete, un token che lo attraversa, un contratto da tenere compatibile.

Efesto

Ogni strato risponde a una **domanda** sola

Strato	Domanda	Pacchetto
Webapp, CLI, agenti	Cosa mostro, quando chiamo?	`webapp-*`
Canale	Come arriva il comando? Sostituibile.	`bus`
Schema dell'input	L'input ha la forma giusta?	`commands`
Middleware: policy	Questo attore può inviare il comando?	`bus`
Handler	Come diventa l'input dello use case?	`bus`
Use case, entity, value object	Il dominio lo consente? Se sì, eseguo.	`core`
Porta → adapter	Dove salvo, a chi invio?	`core` → `adapters/*`

Fino al 2·B forma e permessi li controlla la route. Restano comunque fuori dallo use case.

Efesto

Un oggetto e un contesto in ingresso, un **Result** in uscita



Fa

- Si fida del tipo dichiarato
- Applica le regole tramite entity e value object
- Usa l'attore come dato, non per i permessi
- Ritorna sempre un `Result`

Non fa

- Autenticazione o autorizzazione
- Rivalidare ciò che il tipo garantisce
- Leggere richiesta, cookie, ambiente
- Ricevere argomenti posizionali

Efesto

Uno use case può chiamarne un altro, a **condizioni**

ImportExpenses

```
create.execute(row, ctx)
```

@inject · ICreateExpense →

CreateExpense

stesso contesto, stessa transazione

→ **Dipendenza per interfaccia,** iniettata via DI.

→ **Stesso contesto** passato allo use case chiamato.

→ **Una transazione sola:** l'unit of work è rientrante e vive per richiesta, non come singleton.

→ **Nessun ciclo** fra use case.

Efesto

Chi è l'attore è policy, com'è la risorsa è **dominio**

Policy

Dipende dall'attore rispetto alla risorsa: ruolo, appartenenza, possesso.

«Solo l'autore invia la nota»

Fuori dallo use case: la route fino al 2·B, il middleware del bus al 2·C.

Dominio

Dipende dallo stato della risorsa.

«Solo se è in bozza»

Dentro lo use case, attraverso l'entity.

> *I casi che il criterio non copre li decide l'applicazione.*

Efesto

Si chiama solo **bus.dispatch**, qualunque sia il canale

1

Chiamante

Invia nome e input. Non sa dove gira l'handler.

2

Canale

In-process, Workers RPC, HTTP. Sostituibile.

3

Dispatcher

Valida l'input con lo schema del comando.

4

Middleware

Policy, audit, logging.

5

Handler

Traduce il comando nello use case.

> *Il ritorno è sempre una Promise dell'esito. Il bus non ha code e non differisce niente.*

Efesto

Il comando è il **contratto** fra client e bus

```
// commands/src/approveExpense.ts
export default command({
    input: z.object({
        expense_id: z.string(),
    }),
    output: expenseDto,
    kind: "mutation",
    description: "Approva una nota spese",
    policy: can("expenses.approve"),
});
```

→ **Schema:** il client può validare prima di inviare; fa fede il controllo del dispatcher.

→ **Errori:** con nome, dichiarati nell'output. Il resto arriva come errore interno generico.

→ **Client:** importano `commands` e `core/client`; mai use case, container, adapter.

Nessun comando senza policy

La regola

- La policy è un dato: CLI e agenti la leggono
- Aperta a ogni autenticato: `anyMember`, esplicito
- Nessun accesso anonimo: serve un token valido
- Nessun default: né «nega» né «consenti» se manca

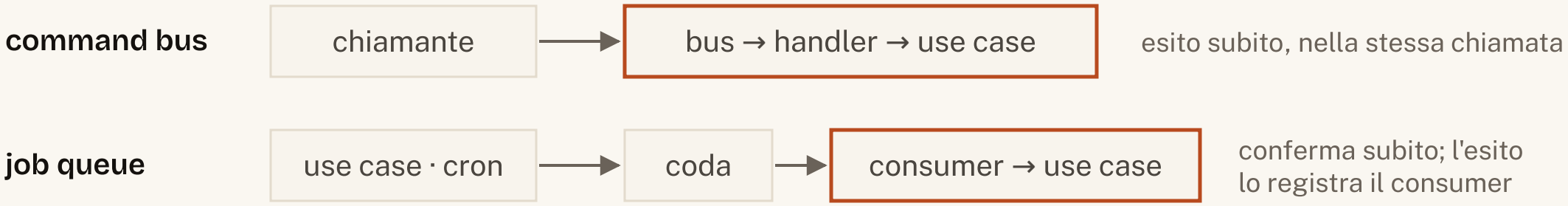
La verifica

- Snapshot versionato di comandi e policy: ogni cambio di permessi è nel diff della PR
- Test di contratto sui due canali, in-process e remoto

Da decidere: dove si dichiara la policy di uno use case. Gli esempi la mettono nel comando.

Efesto

Il bus risponde subito, la coda esegue **dopo**



Il blueprint fissa

- Il lavoro differito passa da una porta del core
- Coda e scheduler sono adapter, con il loro stub
- Il consumer chiama lo use case di sempre

Resta al progetto

- Dove girano consumer e scheduler
- Retry, parallelismo, repliche, idempotenza
- L'attore dei job

Efesto

I principi stanno nella skill, gli strumenti in **stack/**

Principio	Preferenza attuale	Alternative
Repository in un adapter	Prisma	Drizzle, query dirette
Il core non dipende dal runtime	Cloudflare Workers, D1	container, Postgres
Porte iniettate nel costruttore	Inversify	tsyringe, TypeDI
Result con errori con nome	valori di Effect 4	libreria propria, stessa forma
Input validato al bordo	zod	Schema di Effect
Canale sostituibile	Workers RPC	HTTP, altro RPC
Coda e scheduler dietro una porta	Cloudflare Queues, Cron Trigger	pg-boss, croner

Ogni preferenza è una pagina con la data della scelta: cambiare ORM aggiunge una pagina, non modifica le altre.

Efesto

Meno pacchetti: i valori da Effect 4, il resto si **tiene**

Si tiene

entity, usecase (col vincolo sul Result), unit-of-work, expand, metadata, teardown, publisher, observable, react-observable, computed

Si ritira

result, maybe, tagged-error e affini → Effect 4. I pacchetti per ORM → template in `stack/`. login-session, audit.

Da rivedere

toast, da riallineare. env, da far girare dove `process` non c'è. outbox, sospeso con i job.

> *Da Effect solo i valori, non il runtime. Inversify e i decoratori restano, per ora.*

Efesto

Un dominio, cinque cartelle, un **diff** per ogni mossa

```
example/
  0-single/
  1-core/
  2a-adapters/
  2b-bootstrap/
  2c-bus/

# la mossa da 2·B a 2·C
diff -r example/2b-bootstrap \
        example/2c-bus
```

- → **Note spese:** draft → submitted → approved | rejected → reimbursed.
- → **Due scritture:** `ExpenseStatusChange` nella stessa transazione.
- → **Due webapp:** lo staff invia, l'admin approva e rimborsa.
- → **Uno script** fa girare typecheck e test su tutte le cartelle.

Efesto

Prima l'esempio, poi la **skill**

1

lib/example

Le cinque cartelle, con lo script di verifica.

2

SKILL.md

Rivista su questo documento, snippet presi dall'esempio.

3

References

growth, command-bus, testing, stack/*; le altre riscritte.

4

Pulizia

Le skill figlie si rimuovono.

Ancora aperto: dove si dichiara la policy di uno use case.

Efesto

Domande?

efesto-cloud/lib · issue 32